

Low-Code Computer Vision for Precision Livestock Farming

A Hands-On Workshop with Roboflow & Ultralytics

This workshop walks you through the **full computer vision pipeline** for Precision Livestock Farming (PLF) — from labeling images, to training models, to running inference and deploying to edge devices. No prior programming experience is required.

You'll work with two low-code platforms:

-  **Roboflow** — image annotation and dataset preparation
-  **Ultralytics (YOLO11)** — model training, evaluation, and inference

What's in this workshop

File	Description
RoboFlow.pptx	Slide deck — the guided walkthrough used during the session (labeling, training, and deployment, screenshot-by-screenshot).
PLF_ComputerVision_Workshop.ipynb	Google Colab notebook — the hands-on code you run yourself, covering all four vision tasks end to end.

The slides and the notebook are meant to be used **together**: the deck explains the *why* and shows the Roboflow UI steps, while the notebook is where you run the *how*.

What you'll learn

The workshop covers four computer vision tasks, each from labeling all the way through to a trained model:

Task	Roboflow Annotation	Ultralytics Model	Example PLF Use
Image Classification	Single-label tagging	yolo11n-cls	Pig posture (standing / lying)
Object Detection	Bounding boxes	yolo11n	Counting animals in a pen
Instance Segmentation	Polygon masks	yolo11n-seg	Measuring body area / condition
Keypoint Detection	Skeleton points	yolo11n-pose	Pose & gait for lameness



Before you start (prerequisites)

- 1 A free [Roboflow](#) account — sign up *before* the workshop so you're ready to label.
- 2 A **Google account** — the notebook runs in [Google Colab](#), free in the browser, nothing to install locally.
- 3 **Enable the GPU runtime in Colab** — this makes training dramatically faster:
Runtime → Change runtime type → T4 GPU → Save



Getting started

1. Open the slides

Open RoboFlow.pptx in PowerPoint, Google Slides, or Keynote. Follow along here first — the deck shows exactly where to click in the Roboflow interface for each labeling task.

2. Open the notebook in Colab

- Go to colab.research.google.com
- File → Upload notebook and choose
PLF_ComputerVision_Workshop.ipynb
- Enable the GPU runtime (see prerequisites above)
- Run the cells **in order, top to bottom** — later cells depend on variables and models created in earlier ones.

The very first code cell installs everything you need:

```
pip install ultralytics roboflow supervision
```

3. Connect your data

The notebook mounts Google Drive and reads datasets and test media from a Materials folder. Update the file paths near the top of the relevant cells to point at your own dataset (each path to edit is marked with a # ← `update this` comment).



Notebook outline

Section	Topic
1. Environment Setup	Install packages, check the GPU, mount Google Drive

2. Roboflow — Labeling & Dataset Prep	What Roboflow is, plus labeling for classification (2.2), object detection (2.3), instance segmentation (2.4), and keypoints (2.5); exporting your dataset (2.6)
3. Ultralytics — Model Training	What YOLO is, then training a model for each of the four tasks (3.2–3.5)
4. Model Evaluation & Inference	Inference on a single image (4.1), on a video (4.2), and tracking keypoints across video frames (4.3)
5. Edge Deployment	Exporting to deployment formats (5.1) and loading the TFLite model to run inference (5.2)
6. Summary & Next Steps	Recap, resources, and ideas for your own projects

Tips & troubleshooting

- **Run cells in order.** Many cells reuse variables (e.g. `best_det`, `best_pose`, `TEST_IMAGE_DET`) defined earlier. Skipping ahead will cause `NameErrors`.
- **"No GPU detected"** — you forgot to enable the T4 GPU runtime. Set it, then re-run from the top.
- **Training is slow** — that's normal for the first run while data downloads; lower `EPOCHS` for a quick test, or swap `yolo11n` for an even smaller run. Use the bigger `yolo11s` / `yolo11m` variants later for better accuracy.
- **Wrong keypoint is tracked (4.3)** — adjust `KEYPOINT_INDEX`; the index depends on how you ordered the skeleton in Roboflow.
- **TFLite looks *slower* than PyTorch (5.2)** — expected on Colab, because TFLite runs on CPU while the `.pt` model uses the GPU. On an actual edge device (CPU-only) the TFLite model is the lighter, faster choice.
- **First TFLite run pauses to install dependencies** — Ultralytics pulls in extra TensorFlow packages the first time; this only happens once.

Key resources

Resource	Link
Roboflow Documentation	https://docs.roboflow.com
Roboflow Universe (public datasets)	https://universe.roboflow.com
Ultralytics Documentation	https://docs.ultralytics.com
YOLO11 Models	https://docs.ultralytics.com/models/yolo11
Ultralytics GitHub	https://github.com/ultralytics/ultralytics
Google Colab	https://colab.research.google.com
PLF datasets to search on Roboflow Universe: cattle detection, pig behavior,	

livestock body condition,poultry detection,animal lameness.